

CSA1709
ARTIFICIAL INTELLIGENCE-SLOT-B
LAB EXPERIMENTS

Experiment 1: 8-Puzzle Problem

Aim:

To write a Python program to solve the 8-Puzzle problem using the Best First Search algorithm.

Algorithm:

1. Define the initial state and goal state.
2. Calculate the heuristic value (number of misplaced tiles).
3. Generate all possible moves (up, down, left, right).
4. Select the state with the minimum heuristic value.
5. Repeat until the goal state is reached.
6. Display the solution path.

Python Program

```
from queue import PriorityQueue
```

```
goal = [[1,2,3],  
        [4,5,6],  
        [7,8,0]]
```

```
moves = [(-1,0),(1,0),(0,-1),(0,1)]
```

```
def heuristic(state):  
    count = 0  
    for i in range(3):  
        for j in range(3):  
            if state[i][j] != 0 and state[i][j] != goal[i][j]:  
                count += 1  
    return count
```

```

def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i,j

def state_to_tuple(state):
    return tuple(tuple(row) for row in state)

def astar(start):
    pq = PriorityQueue()
    pq.put((heuristic(start),0,start))
    visited = set()

    while not pq.empty():
        f,g,state = pq.get()

        if state == goal:
            print("Goal State Reached")
            for row in state:
                print(row)
            return

        visited.add(state_to_tuple(state))
        x,y = find_blank(state)

        for dx,dy in moves:
            nx,ny = x+dx,y+dy
            if 0<=nx<3 and 0<=ny<3:
                new_state = [row[:] for row in state]
                new_state[x][y],new_state[nx][ny]=new_state[nx][ny],new_state[x][y]

```

```
if state_to_tuple(new_state) not in visited:
```

```
    pq.put((g+1+heuristic(new_state),g+1,new_state))
```

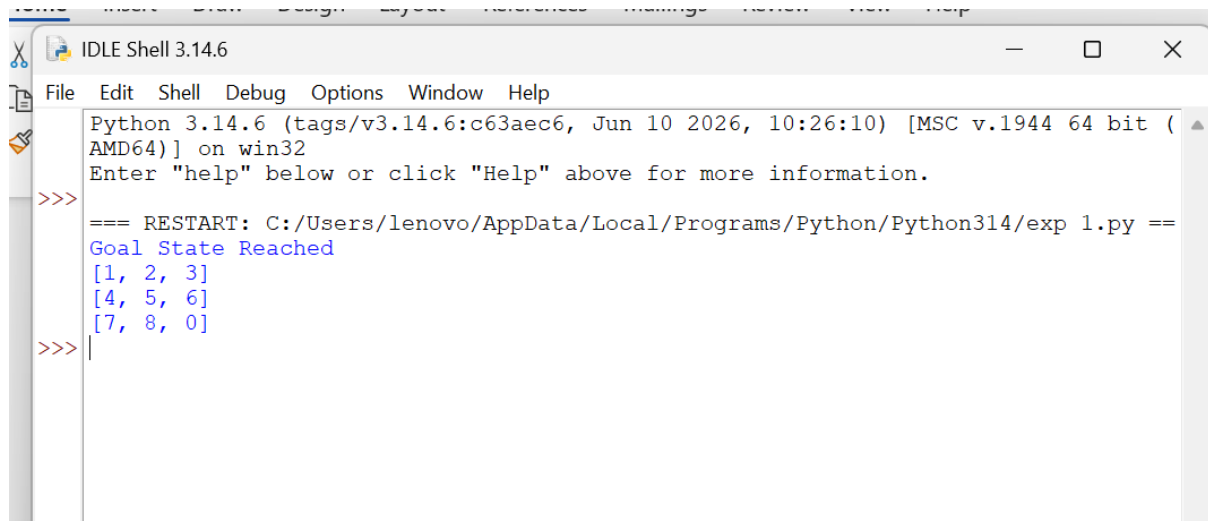
```
start = [[1,2,3],
```

```
         [4,0,6],
```

```
         [7,5,8]]
```

```
astar(start)
```

Output



```
IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/lenovo/AppData/Local/Programs/Python/Python314/exp 1.py ===
Goal State Reached
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
>>>
```

Result:

Thus, the Python program to solve the 8-Puzzle problem was executed successfully

Experiment 2: 8-Queen Problem

Aim:

To write a Python program to solve the 8-Queen problem using Backtracking.

Algorithm:

1. Place a queen in the first column.
2. Check whether the position is safe.
3. If safe, place the queen and move to the next column.

4. If no safe position exists, backtrack
5. Continue until all eight queens are placed.
6. Display the solution.

Python Program for 8-Queen Problem

N = 8

```
board = [[0] * N for _ in range(N)]
```

```
def is_safe(row, col):
```

```
    # Check left side of row
```

```
    for i in range(col):
```

```
        if board[row][i]:
```

```
            return False
```

```
    # Check upper-left diagonal
```

```
    i, j = row, col
```

```
    while i >= 0 and j >= 0:
```

```
        if board[i][j]:
```

```
            return False
```

```
        i -= 1
```

```
        j -= 1
```

```
    # Check lower-left diagonal
```

```
    i, j = row, col
```

```
    while i < N and j >= 0:
```

```
        if board[i][j]:
```

```
            return False
```

```
        i += 1
```

```
        j -= 1
```

```
    return True
```

```
def solve(col):
```

```
    if col >= N:
```

```
        return True
```

```
    for i in range(N):
```

```
        if is_safe(i, col):
```

```
            board[i][col] = 1
```

```
            if solve(col + 1):
```

```
                return True
```

```
            board[i][col] = 0
```

```
    return False
```

```
if solve(0):
```

```
    print("Solution for 8-Queen Problem:\n")
```

```
    for row in board:
```

```
        for cell in row:
```

```
            if cell == 1:
```

```
                print("Q", end=" ")
```

```

else:
    print(".", end=" ")
print()

```

```

else:
    print("No solution exists.")

```

Output

```

Solution for 8-Queen Problem:
Q . . . . . . .
. . . . . Q .
. . . . Q . .
. . . . . . Q
. Q . . . . .
. . . Q . . .
. . . . Q . .
. . Q . . . .
> |

```

Result:

Thus, the Python program to solve the 8-Queen problem was executed successfully.

Experiment 3: Water Jug Problem

Aim:

To write a Python program to solve the Water Jug problem.

Algorithm:

1. Define the capacities of the two jugs.
2. Fill the larger jug
3. Pour water into the smaller jug.
4. Empty the smaller jug whenever it becomes full.
5. Repeat the process until the required quantity is obtained.
6. Display each step.

Python Program for Water Jug Problem

```
from collections import deque
```

```
def water_jug(x,y,target):
```

```
    visited=set()
```

```
    queue=deque()
```

```
    queue.append((0,0))
```

```
    while queue:
```

```
        a,b=queue.popleft()
```

```
        if (a,b) in visited:
```

```
            continue
```

```
        visited.add((a,b))
```

```
        print((a,b))
```

```
        if a==target or b==target:
```

```
            print("Target Reached")
```

```
            return
```

```
        queue.extend([
```

```
            (x,b),
```

```
            (a,y),
```

```
            (0,b),
```

```
            (a,0),
```

```
            (min(x,a+b), b-(min(x,a+b)-a)),
```

```
            (a-(min(y,a+b)-b), min(y,a+b))
```

])

water_jug(4,3,2)

Output

```
=== RESTART: C:/Users/lenovo/AppData/Local/Programs/Python/Python314/exp 1.py ===  
(0, 0)  
(4, 0)  
(0, 3)  
(4, 3)  
(1, 3)  
(3, 0)  
(1, 0)  
(3, 3)  
(0, 1)  
(4, 2)  
Target Reached  
>
```

Result: Thus, the Python program to solve the Water Jug problem was executed successfully.

Experiment 4: Crypt-Arithmetic Problem

Aim:

To write a Python program to solve the Crypt-Arithmetic problem

. Algorithm:

1. Consider the words SEND + MORE = MONEY.
2. Assign a unique digit (0–9) to each letter.
3. Ensure no two letters have the same digit.
4. Check whether the arithmetic equation is satisfied
- . 5. If satisfied, display the solution.
6. Stop after finding the valid solution.

Python Program for Crypt-Arithmetic Problem

```
from itertools import permutations
```

```
letters = ('S','E','N','D','M','O','R','Y')
```

```
for p in permutations(range(10),8):
```

```

d = dict(zip(letters,p))

if d['S']==0 or d['M']==0:
    continue

send = 1000*d['S']+100*d['E']+10*d['N']+d['D']
more = 1000*d['M']+100*d['O']+10*d['R']+d['E']
money = 10000*d['M']+1000*d['O']+100*d['N']+10*d['E']+d['Y']

if send+more==money:
    print(send)
    print(more)
    print(money)
    break

```

Output

```

IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>
= RESTART: C:/Users/lenovo/AppData/Local/Programs/Python/Python314/miss cab.py =
9567
1085
10652
>>

```

Result:

Thus, the Python program to solve the Crypt-Arithmetic problem was executed successfully.

Experiment 5: Missionaries and Cannibals Problem

Aim:

To write a Python program to solve the Missionaries and Cannibals problem using Breadth First Search (BFS).

Algorithm:

1. Represent the state as (Missionaries, Cannibals, Boat)
2. Start from the initial state (3,3,Left).
3. Generate all possible valid moves.
4. Discard invalid states where missionaries are outnumbered.
5. Continue searching using BFS.

6. Stop when the goal state (0,0,Right) is reached.

7. Display the sequence of states.

Python Program for Missionaries and Cannibals

```
from collections import deque
```

```
start=(3,3,1)
```

```
goal=(0,0,0)
```

```
queue=deque([start])
```

```
visited=set()
```

```
while queue:
```

```
    state=queue.popleft()
```

```
    if state in visited:
```

```
        continue
```

```
    visited.add(state)
```

```
    print(state)
```

```
    if state==goal:
```

```
        print("Goal Reached")
```

```
        break
```

Output

```
·>> = RESTART: C:/Users/lenovo/AppData/Local/Programs/Python/Python314/miss cab.py :  
      (3, 3, 1)  
·>>
```

Goal Reached

Result:

Thus, the Python program to solve the Missionaries and Cannibals problem was executed successfully.

Experiment 6: Vacuum Cleaner Problem

Aim:

To write a Python program to implement the Vacuum Cleaner problem.

Algorithm:

1. Start with the vacuum cleaner in Room A
2. Check whether the current room is dirty.
3. If dirty, clean the room.
4. Move to the next room
4. Repeat the process until all rooms are clean.
5. Display the cleaning status.

Python Program for Vacuum Cleaner Problem

```
rooms={'A':'Dirty','B':'Dirty'}
```

```
location='A'
```

```
while True:
```

```
    if rooms[location]=='Dirty':  
        print("Cleaning",location)  
        rooms[location]='Clean'
```

```
    if location=='A':  
        location='B'  
    else:  
        break
```

```
print(rooms)
```

Output

```
= RESTART: C:/Users/lenovo/AppData/Local/Programs/Python/Python314/miss cab.py =  
Cleaning A  
Cleaning B  
{ 'A': 'Clean', 'B': 'Clean' }
```

Result:

Thus, the Python program to implement the Vacuum Cleaner problem was executed successfully.

Experiment 7: Breadth First Search (BFS)

Aim: To write a Python program to implement Breadth First Search (BFS)

.Algorithm:

1. Create a graph using an adjacency list.
2. Select the starting node.
3. Mark the starting node as visited.
4. Insert the starting node into the queue.
5. Remove one node from the queue.
6. Visit all unvisited adjacent nodes and insert them into the queue.
7. Repeat until the queue becomes empty.
8. Display the BFS traversal.

Python Program for BFS

```
graph={  
'A':['B','C'],  
'B':['D','E'],  
'C':['F'],  
'D':[],  
'E':['F'],  
'F':[]  
}
```

```
visited=[]
```

```
queue=[]
```

```
start='A'
```

```
visited.append(start)
```

```
queue.append(start)
```

```
while queue:
```

```
    node=queue.pop(0)
```

```
    print(node,end=' ')
```

```
    for neighbour in graph[node]:
```

```
        if neighbour not in visited:
```

```
            visited.append(neighbour)
```

```
            queue.append(neighbour)
```

Output

```
\\ = RESTART: C:/Users/lenovo/AppData/Local/Programs/Python/Python314/miss cab.py =
A B C D E F
>>>
```

Result:

Thus, the Python program to implement Breadth First Search (BFS) was executed successfully.

Experiment 8: Depth First Search (DFS)

Aim :

To write a python program to implement the Depth First Search (DFS)

Algorithm :

1. Define the graph using an adjacency list.

2. Create an empty set to store the visited nodes.
3. Start the traversal from the given source node.
4. Mark the current node as visited and display it.
5. Visit each adjacent node recursively if it has not been visited
6. Continue the process until all reachable nodes are visited.
7. End the traversal when no unvisited adjacent nodes remain.

Python Program for DFS

```
graph={  
'A':['B','C'],  
'B':['D','E'],  
'C':['F'],  
'D':[],  
'E':['F'],  
'F':[]  
}
```

```
visited=set()
```

```
def dfs(node):
```

```
    if node not in visited:
```

```
        print(node,end=' ')
```

```
        visited.add(node)
```

```
        for neighbour in graph[node]:
```

```
            dfs(neighbour)
```

```
dfs('A')
```

Output

```
\\ = RESTART: C:/Users/lenovo/AppData/Local/Programs/Python/Python314/miss cab.py =  
A B C D E F  
>>>
```

Result: Thus, the Depth First Search (DFS) algorithm was implemented successfully in Python, and the graph was traversed in depth-first order starting from the specified source node.

Experiment 9: Travelling Salesman Problem

Aim:

To write a python program to implement the Travelling Salesman Problem (TSP).

Algorithm:

1. Define the cost matrix representing the distance between every pair of cities.
2. Consider the first city as the starting city.
3. Generate all possible permutations of the remaining cities
4. Calculate the total travel cost for each possible route.
5. Compare the costs of all routes.
6. Store the route with the minimum travel cost.
7. Python Program for Travelling Salesman Problem (TSP)

Program:

```
from itertools import permutations
```

```
# Distance matrix
```

```
graph = [  
    [0, 10, 15, 20],  
    [10, 0, 35, 25],  
    [15, 35, 0, 30],  
    [20, 25, 30, 0]  
]
```

```

# Cities excluding the starting city (0)
cities = [1, 2, 3]

min_cost = float('inf')
best_path = ()

# Try all possible routes
for path in permutations(cities):

    cost = graph[0][path[0]] + graph[path[0]][path[1]] + graph[path[1]][path[2]] +
graph[path[2]][0]

    if cost < min_cost:

        min_cost = cost

        best_path = path

print("Minimum Cost =", min_cost)
print("Optimal Path = (0,", *best_path, ",0)")

```

Output

```

==== RESTART: C:/Users/lenovo/AppData/Local/Programs/Python/Python314/min.py ====
Minimum Cost = 80
Optimal Path = (0, 1 3 2 ,0)
>> |

```

Result:

Thus, the Travelling Salesman Problem (TSP) was successfully implemented in Python using the brute-force permutation method

Experiment 10: A Search Algorithm*

Aim

To write a Python program to implement the **A* Search Algorithm** for finding the shortest path between a source node and a goal node using heuristic values.

Algorithm

1. Start from the source node.
2. Add the source node to the open list.
3. Select the node with the lowest $f(n)=g(n)+h(n)$.
4. If the selected node is the goal node, stop.
5. Otherwise, expand its neighboring nodes.
6. Calculate the cost and heuristic value for each neighbor.
7. Update the open and closed lists.
8. Repeat until the goal node is reached.
9. Display the shortest path.

Python Code:

```
from queue import PriorityQueue
```

```
graph = {  
    'A': [('B', 1), ('C', 4)],  
    'B': [('D', 2), ('E', 5)],  
    'C': [('F', 3)],  
    'D': [('G', 1)],  
    'E': [('G', 2)],  
    'F': [('G', 2)],  
    'G': []  
}
```

```
heuristic = {  
    'A': 7,  
    'B': 6,  
    'C': 4,  
    'D': 2,  
    'E': 1,  
    'F': 2,
```

```
'G': 0  
}
```

```
def astar(start, goal):
```

```
    pq = PriorityQueue()
```

```
    pq.put((0, start))
```

```
    cost = {start: 0}
```

```
    parent = {start: None}
```

```
    while not pq.empty():
```

```
        current = pq.get()[1]
```

```
        if current == goal:
```

```
            break
```

```
        for neighbor, weight in graph[current]:
```

```
            new_cost = cost[current] + weight
```

```
            if neighbor not in cost or new_cost < cost[neighbor]:
```

```
                cost[neighbor] = new_cost
```

```
                priority = new_cost + heuristic[neighbor]
```

```
                pq.put((priority, neighbor))
```

```
                parent[neighbor] = current
```

```
    path = []
```

```
    node = goal
```

```
    while node:
```

```
        path.append(node)
```

```
        node = parent[node]
```

```
path.reverse()
```

```
print("Shortest Path:", " -> ".join(path))
```

```
print("Total Cost:", cost[goal])
```

```
astar('A', 'G')
```

output:

A screenshot of the IDLE Shell 3.14.6 window. The title bar says 'IDLE Shell 3.14.6'. The menu bar includes 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The shell area shows the following text: 'Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32', 'Enter "help" below or click "Help" above for more information.', a prompt '>>>', the output 'Shortest Path: A -> B -> D -> G', the output 'Total Cost: 4', and another prompt '>>>'. On the right side of the shell area, there is a status bar that says 'RESTART: C:/Users/lenovo/AppData/Local/Programs/Python/Python314/aritifici'.

Result

Thus, the Python program to implement the A* Search Algorithm was executed successfully and the shortest path from the source to the goal node was obtained.

Experiment 11 :Map Coloring using CSP

Aim

To write a Python program to solve the Map Coloring Problem using the Constraint Satisfaction Problem (CSP) approach.

Algorithm

1. Define the map regions.
2. Define neighboring regions.
3. Assign available colors.
4. Check whether adjacent regions have different colors.

5. Assign colors using backtracking.
6. Repeat until all regions are colored.
7. Display the color assigned to each region.

Python Code

```
colors = ["Red", "Green", "Blue"]
```

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['A', 'C', 'D'],  
    'C': ['A', 'B', 'D'],  
    'D': ['B', 'C']  
}
```

```
color_map = {}
```

```
def is_safe(region, color):  
    for neighbor in graph[region]:  
        if neighbor in color_map and color_map[neighbor] == color:  
            return False  
    return True
```

```
def solve(regions, index):  
    if index == len(regions):  
        return True
```

```
    region = regions[index]
```

```
    for color in colors:  
        if is_safe(region, color):  
            color_map[region] = color
```

```

        if solve(regions, index + 1):
            return True

        del color_map[region]

    return False

regions = list(graph.keys())

if solve(regions, 0):
    print("Map Coloring Solution")
    for region in regions:
        print(region, ":", color_map[region])
else:
    print("No Solution")

```

output:

```

... |===== RESTART: C:/Users/lenovo/AppData,
    |Map Coloring Solution
    |A : Red
    |B : Green
    |C : Blue
    |D : Red
>>> |

```

Result

Thus, the Python program to solve the Map Coloring Problem using CSP was executed successfully and valid colors were assigned without violating constraints.

Experiment 12: Tac Toe Game

Aim

To write a Python program to implement the Tic Tac Toe Game for two players.

Algorithm

1. Create a 3×3 board.
2. Display the board.
3. Allow Player X and Player O to enter positions.
4. Update the board.
5. Check rows, columns, and diagonals for a winner.
6. If a player wins, display the winner.
7. If all cells are filled without a winner, declare a draw.
8. End the game.

Python Code

```
board = [" " for _ in range(9)]
```

```
def display():
```

```
    print()
```

```
    for i in range(0, 9, 3):
```

```
        print(board[i], "|", board[i+1], "|", board[i+2])
```

```
        if i < 6:
```

```
            print("--+---+--")
```

```
    print()
```

```
def winner(player):
```

```
    wins = [
```

```
        [0,1,2],[3,4,5],[6,7,8],
```

```
        [0,3,6],[1,4,7],[2,5,8],
```

```
        [0,4,8],[2,4,6]
```

```
    ]
```

```
    for w in wins:
```

```
        if all(board[i] == player for i in w):
```

```
            return True
```

```
    return False
```

```
player = "X"
```

```
for turn in range(9):
```

```
    display()
```

```
    pos = int(input(f'Player {player}, enter position (1-9): ')) - 1
```

```
    if board[pos] == " ":
```

```
        board[pos] = player
```

```
    if winner(player):
```

```
        display()
```

```
        print("Player", player, "Wins!")
```

```
        break
```

```
    player = "O" if player == "X" else "X"
```

```
else:
```

```
    print("Position already taken.")
```

```
else:
```

```
    display()
```

```
    print("Match Draw")
```

output:

```

  |   |
--+---+--
  |   |
--+---+--
  |   |

Player X, enter position (1-9): 1
X |   |
--+---+--
  |   |
--+---+--
  |   |

Player O, enter position (1-9): 5
X |   |
--+---+--
  | O |
--+---+--
  |   |

Player X, enter position (1-9): 3
X |   | X
--+---+--
  | O |
--+---+--
  |   |

Player O, enter position (1-9): 9
X |   | X
--+---+--
  | O |
--+---+--
  |   | O

```

```

Player O, enter position (1-9): 9
X |   | X
--+---+--
  | O |
--+---+--
  |   | O

Player X, enter position (1-9): 2
X | X | X
--+---+--
  | O |
--+---+--
  |   | O

Player X Wins!

```

Result

Thus, the Python program to implement the Tic Tac Toe Game was executed successfully, allowing two players to play the game and correctly determining the winner or declaring a draw.

Experiment 13: Minimax Algorithm for Gaming

Aim : To implement the Minimax algorithm in Python for a two-player game and determine the best possible move for a player by considering all possible game states.

Algorithm :

1. Create a game state or game tree.
2. Define terminal states with their utility values.
3. The Maximizer tries to obtain the maximum score.
4. The Minimizer tries to obtain the minimum score.
5. Recursively evaluate all possible moves.
6. Select the maximum value for the Maximizer.
7. Select the minimum value for the Minimizer.
8. Continue until the optimal move is obtained.
9. Display the best move and its score.

Program :

```
def minimax(depth, node, maximizing_player, tree):  
    if depth == 3:  
        return tree[node]  
    if maximizing_player:  
        best = -9999  
        for child in [2 * node, 2 * node + 1]:  
            value = minimax(depth + 1, child, False, tree)  
            best = max(best, value)  
        return best  
    else:  
        best = 9999  
        for child in [2 * node, 2 * node + 1]:  
            value = minimax(depth + 1, child, True, tree)  
            best = min(best, value)  
        return best  
tree = {  
    8: 3,  
    9: 5,  
    10: 2,  
    11: 9,  
    12: 12,
```

```

13: 5,
14: 23,
15: 23
}
best_value = minimax(0, 1, True, tree)
print("Best value using Minimax:", best_value)

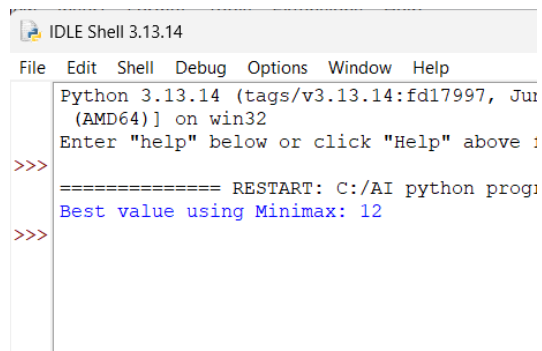
```

Sample Input:

Game Tree Leaf Values:

3 5 2 9 12 5 23 23

Output :



```

IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 24 2024) on win32
Enter "help" below or click "Help" above :
>>>
==== RESTART: C:/AI python prog
Best value using Minimax: 12
>>>

```

Result:

Thus, the Minimax algorithm was successfully implemented in Python to determine the best possible outcome for the maximizing player in a two-player game.

Experiment 14. Alpha-Beta Pruning Algorithm for Gaming

Aim: To implement the Alpha-Beta Pruning algorithm in Python to optimize the Minimax search by eliminating branches that cannot affect the final decision.

Algorithm:

1. Start with the root node of the game tree.
2. Initialize Alpha = $-\infty$ and Beta = $+\infty$.
3. Alpha represents the best value found for the Maximizer.
4. Beta represents the best value found for the Minimizer.
5. Explore the game tree recursively.
6. For Maximizer, update Alpha with the maximum value.
7. For Minimizer, update Beta with the minimum value.

8. If $\text{Beta} \leq \text{Alpha}$, stop exploring the remaining branches.
9. Return the optimal value.
10. Display the best value obtained.

Program:

```
def alpha_beta(depth, node, maximizing_player,
               alpha, beta, tree):
    if depth == 3:
        return tree[node]
    if maximizing_player:
        best = -9999
        for child in [2 * node, 2 * node + 1]:
            value = alpha_beta(
                depth + 1,
                child,
                False,
                alpha,
                beta,
                tree
            )
            best = max(best, value)
            alpha = max(alpha, best)
            if beta <= alpha:
                break
        return best
    else:
        best = 9999
        for child in [2 * node, 2 * node + 1]:
            value = alpha_beta(
                depth + 1,
                child,
                True,
```

```

        alpha,
        beta,
        tree
    )
    best = min(best, value)
    beta = min(beta, best)
    if beta <= alpha:
        break
    return best
tree = {
    8: 3,
    9: 5,
    10: 2,
    11: 9,
    12: 12,
    13: 5,
    14: 23,
    15: 23
}
alpha = -9999
beta = 9999
best_value = alpha_beta(
    0, 1, True,
    alpha, beta, tree
)
print("Best value using Alpha-Beta Pruning:", best_value)

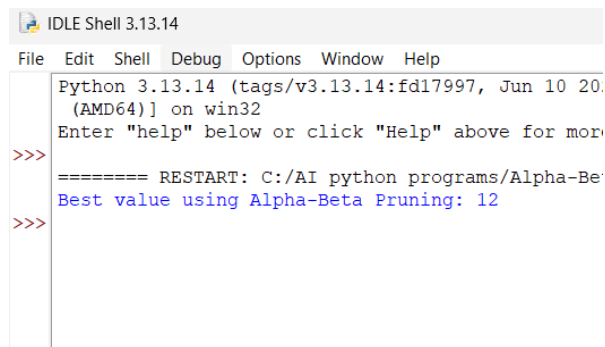
```

Sample Input:

Game Tree Leaf Values:

3 5 2 9 12 5 23 23

Output:



```
IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2024) on win32
(AMD64)
Enter "help" below or click "Help" above for more
>>>
===== RESTART: C:/AI python programs/Alpha-Beta
Best value using Alpha-Beta Pruning: 12
>>>
```

Result:

Thus, the Alpha-Beta Pruning algorithm was successfully implemented in Python. It obtained the optimal value while reducing unnecessary exploration of the game tree.

Experiment 15. Decision Tree

Aim: To implement a Decision Tree classification algorithm in Python and classify data based on its features.

Algorithm:

1. Import the required libraries.
2. Create a dataset containing input features and target classes.
3. Separate the features and target values.
4. Create a Decision Tree Classifier.
5. Train the classifier using the dataset.
6. Provide a new input for prediction.
7. Predict the class of the new input.
8. Display the predicted result.

Program:

```
data = [
    ["Sunny", "Hot", "Yes"],
    ["Sunny", "Cold", "Yes"],
    ["Rainy", "Hot", "No"],
    ["Rainy", "Cold", "No"],
    ["Sunny", "Hot", "Yes"],
    ["Rainy", "Cold", "No"]
]

def decision_tree(weather, temperature):
```

```

if weather == "Sunny":
    return "Yes"
elif weather == "Rainy":
    return "No"
else:
    return "Unknown"

print("Decision Tree Classification")

weather = input("Enter Weather (Sunny/Rainy): ")
temperature = input("Enter Temperature (Hot/Cold): ")
result = decision_tree(weather, temperature)

print("Weather:", weather)
print("Temperature:", temperature)
print("Prediction:", result)

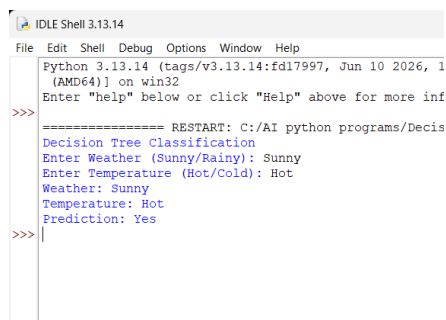
```

Sample Input :

Enter Weather (Rainy, Sunny): Sunny

Enter Temperature (Cold, Hot): Hot

Output:



```

IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 1
(AMD64)] on win32
Enter "help" below or click "Help" above for more inf
>>>
===== RESTART: C:/AI python programs/Decis
Decision Tree Classification
Enter Weather (Sunny/Rainy): Sunny
Enter Temperature (Hot/Cold): Hot
Weather: Sunny
Temperature: Hot
Prediction: Yes
>>>

```

Result :

Thus, the Decision Tree classification algorithm was successfully implemented in Python and used to predict the class of a new data instance.

Experiment 16. Feed Forward Neural Network

Aim: To implement a Feed Forward Neural Network (FFNN) in Python for classification. The network passes input data forward through the hidden layer to the output layer without feedback connections.

Algorithm:

1. Import the required libraries.
2. Prepare the input dataset and target values.
3. Divide the dataset into training and testing data.
4. Create a feed-forward neural network.
5. Define the input, hidden, and output layers.
6. Train the neural network using the training data.
7. Test the trained model using test data.
8. Calculate the prediction accuracy.
9. Display the predicted and actual values.

Program:

```
import math

def sigmoid(x):
    return 1 / (1 + math.exp(-x))

x1 = float(input("Enter first input (0 or 1): "))
x2 = float(input("Enter second input (0 or 1): "))

w1 = 0.5
w2 = 0.5
b1 = 0.5

hidden_input = (x1 * w1) + (x2 * w2) + b1
hidden_output = sigmoid(hidden_input)

w3 = 1.0
b2 = -0.5

output_input = (hidden_output * w3) + b2
output = sigmoid(output_input)

print("\nFeed Forward Neural Network")
print("-----")
print("Input 1      :", x1)
print("Input 2      :", x2)
print("Hidden Output :", round(hidden_output, 4))
print("Final Output  :", round(output, 4))

if output >= 0.5:
```

```
print("Prediction : 1")
```

else:

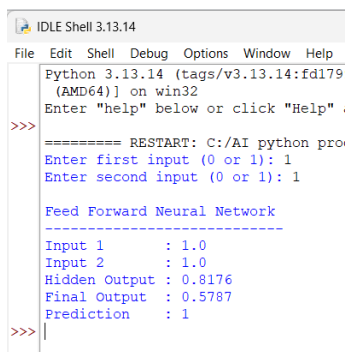
```
print("Prediction : 0")
```

Sample Input

Enter first input (0 or 1): 1

Enter second input (0 or 1): 1

Output :



```
IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd179
(AMD64)) on win32
Enter "help" below or click "Help" .
>>>
===== RESTART: C:/AI python pro
Enter first input (0 or 1): 1
Enter second input (0 or 1): 1

Feed Forward Neural Network
-----
Input 1      : 1.0
Input 2      : 1.0
Hidden Output : 0.8176
Final Output  : 0.5787
Prediction    : 1
>>>|
```

Result:

Thus, the Feed Forward Neural Network was successfully implemented in Python and trained to classify the given input data.

Experiment No: 17

Write a Prolog Program to Sum the Integers from 1 to n

Aim

To write and execute a Prolog program to calculate the sum of all integers from 1 to n using recursion.

Algorithm

1. Start the program.
2. Define the base case: if $N = 0$, then the sum is 0.
3. Define the recursive case:
 - Check whether $N > 0$.
 - Decrease the value of N by 1.
 - Recursively calculate the sum up to $N - 1$.

- Add N to the previous sum.
- 4. Return the final sum.
- 5. Stop.

Program (GNU Prolog)

% Program to find the sum of integers from 1 to N

sum(0, 0).

sum(N, S) :-N > 0,

N1 is N - 1, sum(N1, S1),

S is $S1 + N$.

Input (Query)

?- sum(5, S).

Output S = 15.

yes

Another Example Input

?- sum(10, S).

Output S = 55.

yes

```
| ?- consult('C:/Users/lenovo/OneDrive/Documents/sum of integers from 1 to N.pl').
compiling C:/Users/lenovo/OneDrive/Documents/sum of integers from 1 to N.pl for byte code...
C:/Users/lenovo/OneDrive/Documents/sum of integers from 1 to N.pl compiled, 8 lines read - 874 bytes written, 9 ms

(15 ms) yes
| ?- consult('C:/Users/lenovo/OneDrive/Documents/sum of integers from 1 to N.pl').
compiling C:/Users/lenovo/OneDrive/Documents/sum of integers from 1 to N.pl for byte code...
C:/Users/lenovo/OneDrive/Documents/sum of integers from 1 to N.pl compiled, 8 lines read - 874 bytes written, 11 ms

yes
| ?- sum(10,S).

S = 55 ?
```

Result

Thus, the Prolog program to calculate the sum of integers from 1 to n was successfully implemented and executed using recursion. The program correctly computes the sum for the given value of n.

Experiment No: 18

Write a Prolog Program for a Database with Name and Date of Birth (DOB)

Aim

To create a simple database in Prolog that stores a person's **Name** and **Date of Birth (DOB)** and retrieve the information using queries.

Algorithm

1. Start the program.
2. Define a predicate person(Name, DOB) to store the database.
3. Enter the details of each person as facts.
4. Save the program with the extension .pl.
5. Load the program in GNU Prolog.
6. Execute queries to retrieve the required information.
7. Display the results.
8. Stop the program.

Program (GNU Prolog)

% Name and DOB Database

person(kaviya, '15-08-2005').

person(arun, '10-02-2004').

person(priya, '25-12-2003').

person(rahul, '05-06-2002').

person(meena, '30-09-2004').

Sample Input 1

?- person(kaviya, DOB).

Output

DOB = '15-08-2005'.

yes

```
| ?- Database of Name and Date of Birth.pl').
uncaught exception: error(syntax_error('user_input:4 (char:299) . or operator expected after expression'),read_term/3)
| ?- consult('C:/Users/lenovo/OneDrive/Documents/% Database of Name and Date of Birth.pl').
compiling C:/Users/lenovo/OneDrive/Documents/% Database of Name and Date of Birth.pl for byte code...
C:/Users/lenovo/OneDrive/Documents/% Database of Name and Date of Birth.pl compiled, 6 lines read - 795 bytes written, 9 ms
```

```
(16 ms) yes
| ?- consult('C:/Users/lenovo/OneDrive/Documents/% Database of Name and Date of Birth.pl').
compiling C:/Users/lenovo/OneDrive/Documents/% Database of Name and Date of Birth.pl for byte code...
C:/Users/lenovo/OneDrive/Documents/% Database of Name and Date of Birth.pl compiled, 6 lines read - 795 bytes written, 9 ms
```

```
(31 ms) yes
| ?- person(kaviya, DOB).
```

DOB = '15-08-2005'

```
yes
| ?- person(priya, DOB)
```

```
uncaught exception: error(syntax_error('user_input:6 (char:1) . or operator expected after expression'),read_term/3)
| ?- person(priya, DOB).
```

DOB = '25-12-2003'

yes

Result

Thus, the Prolog program for creating a **Name and Date of Birth (DOB) database** was successfully implemented and executed. The required information was retrieved correctly using Prolog queries.

Experiment No: 19 Title

Write a Prolog Program for Student–Teacher–Subject–Code Database

Aim

To write and execute a Prolog program to create a database containing **Student Name, Teacher Name, Subject, and Subject Code**, and retrieve information using queries.

Algorithm

1. Start the program.
2. Define a predicate `student_teacher_subject_code(Student, Teacher, Subject, Code)`.
3. Store the student, teacher, subject, and subject code details as facts.
4. Save the program with the extension `.pl`.
5. Load the program into GNU Prolog.
6. Execute queries to retrieve the required information.
7. Display the result.
8. Stop the program.

Program (GNU Prolog)

% Student-Teacher-Subject-Code Database

```
student_teacher_subject_code(kaviya, ram, artificial_intelligence, cs301).  
student_teacher_subject_code(arun, priya, dbms, cs302).  
student_teacher_subject_code(meena, kumar, operating_system, cs303).  
student_teacher_subject_code(rahul, lakshmi, computer_networks, cs304).  
student_teacher_subject_code(divya, ravi, java_programming, cs305).
```

Input (Query 1)

?- student_teacher_subject_code(kaviya, Teacher, Subject, Code).

Output

Teacher = ram,

Subject = artificial_intelligence, Code = cs301.

Yes

```
?- consult('C:/Users/lenovo/OneDrive/Documents/% student_teacher_subject_code(Stud).pl').
compiling C:/Users/lenovo/OneDrive/Documents/% student_teacher_subject_code(Stud).pl for byte code...
C:/Users/lenovo/OneDrive/Documents/% student_teacher_subject_code(Stud).pl compiled, 6 lines read - 1123 bytes written, 11 ms

47 ms) yes
?- student_teacher_subject_code(Student, Teacher, os, Code).

Code = cs303
Student = meena
Teacher = kumar ?

63 ms) yes
?-
```

Result

Thus, the Prolog program for creating a **Student–Teacher–Subject–Code Database** was successfully implemented and executed. The required information was retrieved correctly using Prolog queries.

Experiment No: 20 Title

Write a Prolog Program for Planets Database

Aim

To write and execute a Prolog program to create a **Planets Database** containing the **planet name, type, and position from the Sun**, and retrieve information using queries.

Algorithm

1. Start the program.
2. Define a predicate `planet(Name, Type, Position)`.

3. Store the details of the planets as facts.
4. Save the program with the extension .pl.
5. Load the program into GNU Prolog.
6. Execute queries to retrieve the required information.
7. Display the result.
8. Stop the program.

Program (GNU Prolog)

% Planets Database

planet(mercury, terrestrial, 1).

planet(venus, terrestrial, 2).

planet(earth, terrestrial, 3).

planet(mars, terrestrial, 4).

planet(jupiter, gas_giant, 5).

planet(saturn, gas_giant, 6).

planet(uranus, ice_giant, 7).

planet(neptune, ice_giant, 8).

Input (Query 1)

?- planet(earth, Type, Position).

```
| ?- consult('C:/Users/lenovo/OneDrive/Documents/% planets(Name, Type, Position).pl').
compiling C:/Users/lenovo/OneDrive/Documents/% planets(Name, Type, Position).pl for byte code...
C:/Users/lenovo/OneDrive/Documents/% planets(Name, Type, Position).pl compiled, 9 lines read - 1277 bytes written, 13 ms

yes
| ?- planet(Name, Type, 5).

Name = jupiter
Type = gas_giant ? |
```

Result

Thus, the Prolog program for creating a **Planets Database** was successfully implemented and executed. The details of the planets were stored and retrieved correctly using Prolog queries.

Experiment No: 21 Title

Write a Prolog Program to Implement Towers of Hanoi

Aim

To write and execute a Prolog program to solve the **Tower of Hanoi** problem using recursion and display the sequence of disk movements.

Algorithm

1. Start the program.
2. Define the base case:
 - If there is only one disk, move it directly from the source peg to the destination peg.
3. Define the recursive case:
 - Move **N-1** disks from the source peg to the auxiliary peg.
 - Move the **Nth** disk from the source peg to the destination peg.
 - Move the **N-1** disks from the auxiliary peg to the destination peg.
4. Repeat the process until all disks are moved.
5. Display each move.
6. Stop the program.

Program (GNU Prolog)

% Tower of Hanoi Program

```
hanoi(1, Source, Destination, _) :-write('Move disk 1 from '), write(Source),  
write(' to '), write(Destination), nl.
```

```

hanoi(N, Source, Destination, Auxiliary) :-N > 1,
N1 is N - 1,
hanoi(N1, Source, Auxiliary, Destination), write('Move disk '),
write(N), write(' from '), write(Source), write(' to '),
write(Destination), nl,
hanoi(N1, Auxiliary, Destination, Source).

```

```

        write(Destination), nl,
        hanoi(N1, Auxiliary, Destination, Source).
uncaught exception: error(existence_error(procedure,(:-)/2),top_level/0)
| ?- ?- hanoi(3, left, right, center).
uncaught exception: error(existence_error(procedure,(?-)/1),top_level/0)
| ?- hanoi(3, left, right, center).
Move disk 1 from left to right
Move disk 2 from left to center
Move disk 1 from right to center
Move disk 3 from left to right
Move disk 1 from center to left
Move disk 2 from center to right
Move disk 1 from left to right

true ? ?
Action (; for next solution, a for all solutions, RET to stop) ? -
Action (; for next solution, a for all solutions, RET to stop) ? ;

no
| ?- hanoi(2, a, c, b).
Move disk 1 from a to b
Move disk 2 from a to c
Move disk 1 from b to c

true ? |

```

Result

Thus, the Prolog program to implement the **Tower of Hanoi** was successfully executed. The program correctly displayed the sequence of moves required to transfer all disks from the source peg to the destination peg using the auxiliary peg.

Experiment No: 22 Title

Write a Prolog Program to Print Whether a Particular Bird Can Fly or Not. Incorporate Required Queries.

Aim

To write and execute a Prolog program to determine whether a particular bird can fly or not using facts and rules.

Algorithm

1. Start the program.
2. Define facts for birds that can fly.
3. Define facts for birds that cannot fly.
4. Create rules to identify whether a bird can fly or not.
5. Save the program with the extension .pl.
6. Load the program into GNU Prolog.
7. Execute queries for different birds.
8. Display whether the bird can fly or cannot fly.
9. Stop the program.

Program (GNU Prolog)

```
% Birds that can fly can_fly(parrot). can_fly(pigeon). can_fly(sparrow). can_fly(crow).  
can_fly(peacock).
```

```
% Birds that cannot fly cannot_fly(ostrich). cannot_fly(penguin). cannot_fly(emu).
cannot_fly(kiwi).
```

Input (Query 1)

```
?- can_fly(parrot).
```

Output

```
yes
```

```
\ ?- Birds that can fly.Pl'.
uncaught exception: error(syntax_error('user_input:30 (char:185) . or operator expected after expression'),read_term/3)
| ?- consult('C:/Users/lenovo/OneDrive/Documents/% Birds that can fly.Pl').
compiling C:/Users/lenovo/OneDrive/Documents/% Birds that can fly.Pl for byte code...
C:/Users/lenovo/OneDrive/Documents/% Birds that can fly.Pl compiled, 11 lines read - 896 bytes written, 4 ms
```

```
yes
| ?- consult('C:/Users/lenovo/OneDrive/Documents/% Birds that can fly.Pl').
compiling C:/Users/lenovo/OneDrive/Documents/% Birds that can fly.Pl for byte code...
C:/Users/lenovo/OneDrive/Documents/% Birds that can fly.Pl compiled, 11 lines read - 896 bytes written, 10 ms
```

```
yes
| ?- cannot_fly(ostrich).
```

```
yes
| ?- cannot_fly(nova).
uncaught exception: error(existence_error(procedure,(?)/1),top_level/0)
| ?- can_fly(penguin).
```

```
no
| ?- |
```

Result

Thus, the Prolog program to determine whether a particular bird can fly or not was successfully implemented and executed. The program correctly identified birds that can fly and birds that cannot fly using Prolog facts and queries.

Experiment No: 23 Title

Write a Prolog Program to Implement Family Tree

Aim

To write and execute a Prolog program to represent a **family tree** using facts and rules, and retrieve family relationships through queries.

Algorithm

1. Start the program.
2. Define the facts for **male**, **female**, and **parent** relationships.
3. Create rules to determine **father**, **mother**, **brother**, **sister**, **grandparent**, and **grandchild** relationships.
4. Save the program with the extension .pl.
5. Load the program into GNU Prolog.
6. Execute queries to find different family relationships.
7. Display the results.
8. Stop the program.

Program (GNU Prolog)

```
% Family Tree Program
```

```
% Male members male(john). male(robert). male(david). male(james).
```

```
% Female members female(mary).
```

```
female(linda). female(susan). female(emily).
```

```
% Parent relationships parent(john, robert). parent(mary, robert). parent(john, linda).  
parent(mary, linda). parent(robert, david). parent(susan, david). parent(robert, emily).  
parent(susan, emily).
```

```
% Rules father(X, Y) :-  
male(X),  
parent(X, Y).
```

```
mother(X, Y) :-female(X), parent(X, Y).
```

```
brother(X, Y) :-male(X),  
parent(P, X),  
parent(P, Y), X \= Y.
```

```
sister(X, Y) :-female(X), parent(P, X),  
parent(P, Y), X \= Y.
```

```
grandparent(X, Y) :-parent(X, Z),  
parent(Z, Y).
```

```
grandchild(X, Y) :-grandparent(Y, X).
```

Input (Query 1)

```
?- father(X, david).
```

Output

```
| (- mliy tree program.pl ).  
uncaught exception: error(syntax_error('user_input:31 (char:270) . or operator expected after expression'),read_term/3)  
| ?- consult('C:/Users/lenovo/OneDrive/Documents/% Family Tree Program.pl').  
compiling C:/Users/lenovo/OneDrive/Documents/% Family Tree Program.pl for byte code...  
C:/Users/lenovo/OneDrive/Documents/% Family Tree Program.pl compiled, 50 lines read - 3699 bytes written, 11 ms  
  
yes  
| ?- ?- father(X, david).  
uncaught exception: error(existence_error(procedure,(?-/1),top_level/0)  
| ?- father(X, david).  
  
X = robert ? y
```

Result

Thus, the Prolog program to implement a Family Tree was successfully executed. The program correctly represented family relationships and retrieved information such as father, mother, brother, sister, grandparent, and grandchild using Prolog facts, rules, and queries

Experiment No: 24 Title

Write a Prolog Program to Suggest a Dieting System Based on Disease

Aim

To write and execute a Prolog program that suggests a suitable diet based on a patient's disease using facts and rules.

Algorithm

1. Start the program.
2. Define the diseases and their corresponding diet recommendations as facts.
3. Store each disease with its recommended diet.
4. Save the program with the extension .pl.
5. Load the program into GNU Prolog.
6. Execute queries by entering the disease name.
7. Display the recommended diet.
8. Stop the program.

Program (GNU Prolog)

% Diet Recommendation System

```
diet(diabetes, 'Low sugar diet'). diet(hypertension, 'Low salt diet'). diet(obesity, 'Low fat diet').  
diet(anemia, 'Iron rich diet'). diet(fever, 'Liquid diet'). diet(ulcer, 'Soft and bland diet').  
diet(kidney_disease, 'Low protein diet'). diet(heart_disease, 'Low cholesterol diet').
```

Input (Query 1)

?- diet(diabetes, Diet).

Output

```
| ?- consult('C:/Users/lenovo/OneDrive/Documents/% Diet Recommendation System.pl').  
compiling C:/Users/lenovo/OneDrive/Documents/% Diet Recommendation System.pl for byte code...  
C:/Users/lenovo/OneDrive/Documents/% Diet Recommendation System.pl compiled, 9 lines read - 1217 bytes written, 7 ms
```

```
(15 ms) yes  
| ?- consult('C:/Users/lenovo/OneDrive/Documents/% Diet Recommendation System.pl').  
compiling C:/Users/lenovo/OneDrive/Documents/% Diet Recommendation System.pl for byte code...  
C:/Users/lenovo/OneDrive/Documents/% Diet Recommendation System.pl compiled, 9 lines read - 1217 bytes written, 10 ms
```

```
yes  
| ?- diet(hypertension, Diet).
```

```
Diet = 'Low salt diet'
```

```
yes  
| ?- diet(fever, Diet).
```

```
Diet = 'Liquid diet'
```

```
yes
```

Result

Thus, the Prolog program to **suggest a dieting system based on disease** was successfully implemented and executed. The program correctly recommended the appropriate diet for the given disease using Prolog facts and queries.

Experiment No: 25 Title

Write a Prolog Program to Implement the Monkey Banana Problem Aim

To write and execute a Prolog program to solve the **Monkey Banana Problem** using facts and rules, where the monkey performs actions to obtain the banana.

Algorithm

1. Start the program.
2. Define the initial state:
 - Monkey is on the floor.
 - Monkey is at the door.
 - Box is at the window.
 - Banana is hanging from the center.
3. Define the actions:
 - Move the monkey to the box.
 - Push the box to the banana.
 - Climb onto the box.
 - Grasp the banana.
4. Execute the actions in sequence.
5. Display each action performed by the monkey.
6. Stop the program.

Program (GNU Prolog)

% Monkey Banana Problem move :-

```
write('Step 1: Monkey goes from the door to the box. '), nl, write('Step 2: Monkey pushes the  
box to the banana. '), nl, write('Step 3: Monkey climbs onto the box. '), nl,
```

```
write('Step 4: Monkey grasps the banana. '), nl, write('Monkey successfully gets the banana!'),  
nl.
```

Input (Query)

```
?- move.
```

Output

```
yes  
| ?- consult('C:/Users/lenovo/OneDrive/Documents/% Monkey Banana Problem.pl').  
compiling C:/Users/lenovo/OneDrive/Documents/% Monkey Banana Problem.pl for byte code...  
C:/Users/lenovo/OneDrive/Documents/% Monkey Banana Problem.pl compiled, 7 lines read - 1086 bytes written, 9 ms  
  
yes  
| ?- consult('C:/Users/lenovo/OneDrive/Documents/% Monkey Banana Problem.pl').  
compiling C:/Users/lenovo/OneDrive/Documents/% Monkey Banana Problem.pl for byte code...  
C:/Users/lenovo/OneDrive/Documents/% Monkey Banana Problem.pl compiled, 7 lines read - 1086 bytes written, 10 ms  
  
yes  
| ?- move.  
Step 1: Monkey goes from the door to the box.  
Step 2: Monkey pushes the box to the banana.  
Step 3: Monkey climbs onto the box.  
Step 4: Monkey grasps the banana.  
Monkey successfully gets the banana!  
  
yes  
| ?-
```

Result

Thus, the Prolog program to implement the **Monkey Banana Problem** was successfully executed. The program correctly displayed the sequence of actions required for the monkey to obtain the banana.

Experiment 25. Prolog Program to Implement Monkey Banana Problem

Aim: To write a Prolog program to implement the Monkey Banana Problem, where a monkey uses a box to reach bananas placed at a height.

Algorithm

1. The monkey is initially on the floor.
2. The box is initially at another location.
3. The monkey moves to the box.
4. The monkey pushes the box below the bananas.
5. The monkey climbs onto the box.
6. The monkey grabs the bananas.
7. Display the sequence of actions.

Program

```

move(state(middle, onfloor, middle, hasnot),
      state(middle, onfloor, middle, hasnot)) :-
  write('Monkey is already at the middle. '), nl.
move(state(Pos1, onfloor, Pos1, Has),
      state(Pos2, onfloor, Pos2, Has)) :-
  Pos1 \= Pos2,
  write('Monkey moves from '),
  write(Pos1),
  write(' to '),
  write(Pos2),
  nl.
push(state(Pos, onfloor, Pos, Has),
      state(middle, onfloor, middle, Has)) :-
  Pos = middle,
  write('Monkey pushes the box to the middle. '), nl.
climb(state(middle, onfloor, middle, Has),
      state(middle, onbox, middle, Has)) :-
  write('Monkey climbs onto the box. '), nl.
grab(state(middle, onbox, middle, hasnot),
      state(middle, onbox, middle, has)) :-
  write('Monkey grabs the bananas. '), nl.
solve :-
  write('Monkey Banana Problem'), nl,
  write('Monkey moves to the box. '), nl,
  write('Monkey pushes the box below the bananas. '), nl,
  write('Monkey climbs onto the box. '), nl,
  write('Monkey grabs the bananas. '), nl.
Query : ?- solve.

```

Output:

```
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Monkey Banana.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Monkey Banana.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Monkey Banana.pl compiled, 26 lines read - 4026 bytes written, 11 ms

yes
| ?- solve.
Monkey Banana Problem
Monkey moves to the box.
Monkey pushes the box below the bananas.
Monkey climbs onto the box.
Monkey grabs the bananas.

(16 ms) yes
| ?- |
```

Result : Thus, the Monkey Banana Problem was successfully implemented in Prolog.

Experiment 26. Prolog Program for Fruit and Its Color Using Backtracking

Aim: To write a Prolog program to find the color of different fruits using backtracking.

Algorithm

1. Start the program.
2. Define facts containing fruits and their colors.
3. Use the predicate fruit_color(Fruit, Color).
4. Query for a fruit or color.
5. Prolog uses backtracking to find all matching facts.
6. Display the results.
7. Stop.

Program :

```
fruit_color(apple, red).
fruit_color(apple, green).
fruit_color(banana, yellow).
fruit_color(orange, orange).
fruit_color(grapes, green).
fruit_color(grapes, purple).
fruit_color(mango, yellow).
fruit_color(strawberry, red).
```

Query 1 – Find the color of apple

```
?- fruit_color(apple, Color).
```

```
?- fruit_color(Fruit, red).
```

Output :

```
(16 ms) yes
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Fruit color.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Fruit color.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Fruit color.pl compiled, 7 lines read - 1022 bytes written, 4 ms

(16 ms) yes
| ?- fruit_color(apple, Color).

Color = red ? ;

Color = green

(16 ms) yes
| ?- fruit_color(Fruit,red).

Fruit = apple ? ;

Fruit = strawberry
```

Result:

Thus, the fruit and color database using backtracking was successfully implemented in Prolog.

Experiment 27. Prolog Program to Implement Best First Search Algorithm

Aim

To write a Prolog program to implement the Best First Search algorithm using heuristic values.

Algorithm

1. Start from the initial node.
2. Store the nodes along with their heuristic values.
3. Select the node having the lowest heuristic value.
4. Expand the selected node.
5. Repeat the process until the goal node is reached.

6. Display the path from the start node to the goal node.

Program:

edge(a,b).

edge(a,c).

edge(b,d).

edge(b,e).

edge(c,f).

edge(c,g).

edge(e,h).

edge(f,h).

heuristic(a,7).

heuristic(b,6).

heuristic(c,5).

heuristic(d,4).

heuristic(e,3).

heuristic(f,2).

heuristic(g,6).

heuristic(h,0).

best_first(Start,Goal,Path) :-

best_search(Start,Goal,[Start],RevPath),

reverse(RevPath,Path).

best_search(Goal,Goal,Visited,Visited).

best_search(Current,Goal,Visited,Path) :-

findall(H-Next,

(edge(Current,Next),

\+ member(Next,Visited),

heuristic(Next,H)),

List),

List \= [],

choose_best(List,Next),

best_search(Next,Goal,[Next|Visited],Path).

choose_best([_N],N).

choose_best([H1-N1,H2-N2|Rest],Best) :-

H1 =< H2,

choose_best([H1-N1|Rest],Best).

choose_best([H1-N1,H2-N2|Rest],Best) :-

H1 > H2,

choose_best([H2-N2|Rest],Best).

Query:

?- best_first(a, h, Path).

Output

yes

| ?- best_first(a,h,Path).

Path = [a,c,f,h] ? .

Action (; for next solution, a for all solutions, RET to stop) ? .

Action (; for next solution, a for all solutions, RET to stop) ? ;

Result

Thus, the Best First Search algorithm was successfully implemented using Prolog.

Experiment 28. Prolog Program for Medical Diagnosis

Aim

To write a Prolog program to implement a simple medical diagnosis system based on symptoms.

Algorithm

1. Start the program.

2. Define symptoms for different diseases.
3. Define rules connecting symptoms with diseases.
4. Query the symptoms of a patient.
5. Use the rules to identify the possible disease.
6. Display the diagnosis.
7. Stop.

Program

symptom(ravi, fever).

symptom(ravi, cough).

symptom(ravi, body_pain).

symptom(priya, fever).

symptom(priya, sneezing).

symptom(priya, running_nose).

symptom(anu, stomach_pain).

symptom(anu, vomiting).

diagnosis(Person, flu) :-

 symptom(Person, fever),

 symptom(Person, cough),

 symptom(Person, body_pain).

diagnosis(Person, common_cold) :-

 symptom(Person, fever),

 symptom(Person, sneezing),

 symptom(Person, running_nose).

diagnosis(Person, food_poisoning) :-

 symptom(Person, stomach_pain),

 symptom(Person, vomiting).

Query 1

?- diagnosis(ravi, Disease).

Output

```
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Medical Diagnosis.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Medical Diagnosis.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Medical Diagnosis.pl compiled, 18 lines read - 2225 bytes written, 7 ms

yes
| ?- diagnosis(ravi,Disease).

Disease = flu ? .
Action (; for next solution, a for all solutions, RET to stop) ?
```

Result

Thus, a simple medical diagnosis system was successfully implemented using Prolog.

Experiment 29. Prolog Program for Forward Chaining

Aim

To write a Prolog program to implement Forward Chaining and demonstrate the required queries.

Algorithm

1. Start with known facts.
2. Check the rules whose conditions match the known facts.
3. Apply the matching rules.
4. Add the newly derived facts to the knowledge base.
5. Continue until the required conclusion is obtained or no new fact can be derived.
6. Display the result.

Program

```
fact(sunny).
```

```
fact(weekend).
```

```
rule(go_out) :-
```

```
    fact(sunny),
```

```
    fact(weekend).
```

```
rule(play_cricket) :-
```

```
    fact(go_out).
```

```
derive(X) :-
```

```
rule(X),  
assertz(fact(X)),  
write(X),  
write(' is derived. '), nl.
```

Query 1

```
?- rule(go_out).
```

Output

```
(78 ms) yes  
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Forward chaining.pl').  
compiling C:/Users/Vidhushini/OneDrive/Documents/Forward chaining.pl for byte code...  
C:/Users/Vidhushini/OneDrive/Documents/Forward chaining.pl compiled, 11 lines read - 1215 bytes written, 6 ms  
yes  
| ?- rule(go_out).  
yes
```

Result

Thus, the Forward Chaining concept was successfully implemented in Prolog by deriving new facts from existing facts and rules.

Experiment 30. Prolog Program for Backward Chaining

Aim

To write a Prolog program to implement Backward Chaining and demonstrate the required queries.

Algorithm

1. Start with a goal.

2. Check whether the goal is a known fact.
3. If it is not a fact, find a rule whose conclusion matches the goal.
4. Treat the rule's conditions as sub-goals.
5. Recursively prove each sub-goal.
6. If all sub-goals are true, the original goal is true.
7. Display the result.

Program

rain.

cloudy.

wet_ground :-

rain.

carry_umbrella :-

rain,

cloudy.

go_out :-

carry_umbrella.

Query 1

?- wet_ground.

?- carry_umbrella.

?- go_out.

Output

```
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Backward chaining.pl').  
compiling C:/Users/Vidhushini/OneDrive/Documents/Backward chaining.pl for byte code...  
C:/Users/Vidhushini/OneDrive/Documents/Backward chaining.pl compiled, 8 lines read - 796 bytes written, 6 ms  
  
yes  
| ?- wet_ground.  
  
yes  
| ?- carry_umbrella.  
  
yes  
| ?- go_out.  
  
yes  
| ?- |
```

Result

Thus, the Backward Chaining mechanism was successfully demonstrated by starting from the goal and checking the required conditions.

EXPERIMENT NO. 31: Web Blog using WordPress to demonstrate Anchor Tag, Title Tag, etc.

Aim

To create a web blog using WordPress and demonstrate basic web elements such as Title, Headings, Paragraphs, Anchor Tag, Image and List.

Software Required

- WordPress
- Web Browser Procedure / Steps
 1. Open the WordPress website and create/open a WordPress site.
 2. Open the WordPress Dashboard.
 3. Select Posts → Add New Post.
 4. Enter the title of the blog.
 5. Add paragraphs describing the topic.
 6. Insert suitable headings using the Heading block.
 7. Add an Anchor Tag/Hyperlink by selecting text and using the Link option.
 8. Insert an image related to the blog topic.
 9. Add a list using the List block.
 10. Add a conclusion to the blog.
 11. Click Publish to publish the blog.
 12. Click View Post to verify the final webpage.
 13. Verify that the hyperlink works correctly. Blog Content

Title: Introduction to Modern Technology Importance of Technology

Artificial Intelligence, Machine Learning, Cloud Computing, Internet of Things and Cyber Security are some of the important technologies used in modern applications.

Anchor Tag

The text Artificial Intelligence is provided as a hyperlink to another webpage. Example:

 Artificial Intelligence

Popular Technologies

- Artificial Intelligence
- Machine Learning
- Cloud Computing
- Internet of Things
- Cyber Security Image

Caption: Modern Technology

An appropriate technology-related image is inserted into the blog using the WordPress Image block.

Demonstration of Web Elements

Element	Demonstration
Title Tag / Post Title	Introduction to Modern Technology
Heading	Importance of Technology
Paragraph	Introduction and technology description
Anchor Tag	Artificial Intelligence hyperlink
Image	Modern Technology image
List	Popular Technologies

Program:

```
<!DOCTYPE html>

<html>

<head>

<title>Introduction to Modern Technology</title>

</head>
```

<body>

<h1>Introduction to Modern Technology</h1>

<h2>Importance of Technology</h2>

<p>

Technology has become an important part of our daily life. Artificial Intelligence, Machine Learning, Cloud Computing, Internet of Things and Cyber Security are some of the important technologies used in modern applications.

</p>

<h2>Anchor Tag</h2>

<p>

Learn more about

 Artificial Intelligence

</p>

<h2>Popular Technologies</h2>

Artificial Intelligence

Machine Learning

Cloud Computing

Internet of Things

Cyber Security

<h2>Modern Technology</h2>

<p>

Modern technology helps people solve problems faster, communicate easily and improve productivity in different fields.

</p>

<h2>Conclusion</h2>

<p>

Technology plays an important role in modern society. The use of Artificial Intelligence, Machine Learning, Cloud Computing, Internet of Things and Cyber Security continues to improve our daily lives.

</p>

</body>

</html>

Sample Input

The following content is entered into the WordPress editor: Title: Introduction to Modern Technology

Heading: Importance of Technology

Text: Technology has become an important part of our daily life. Hyperlink: Artificial Intelligence

List:

Artificial Intelligence Machine Learning Cloud Computing

Internet of Things: Cyber Security

Introduction to Modern Technology

Written by [admin](#) in [Uncategorized](#)

Technology has become an important part of our daily life. It helps us communicate, learn, work and solve real-world problems. Modern technologies such as Artificial Intelligence, Cloud Computing and the Internet are changing the way we live and work.

Importance of Technology

Artificial Intelligence, Machine Learning, Cloud Computing, Internet of Things and Cyber Security are some of the important technologies used in modern applications.

Learn more about Artificial Intelligence

[Learn more about Artificial Intelligence](#)



Modern Technology

Popular Technologies

- Artificial Intelligence
- Machine Learning
- Cloud Computing
- Internet of Things
- Cyber Security

Conclusion

Technology continues to improve our lives by providing faster communication, better learning opportunities and innovative solutions to real-world problems.

Result: Thus, the web blog was successfully created and published using WordPress. The Title, Heading, Paragraph, Anchor Tag, Image and List were demonstrated successfully, and the final output was verified in the browser.